

friction coefficients, collision filtering information and whatever other information the game might require.

For simple convex primitives, the collision properties are usually associated with the shape as a whole. For polygon soup shapes, the properties might be specified on a per-triangle basis. Because of this latter usage, we usually try to keep the binding between the collision primitive and its collision material as compact as possible. A typical approach is to bind collision primitives to collision materials via an 8-, 16- or 32-bit integer, or a pointer to the material data. This integer indexes into a global array of data structures containing the detailed collision properties themselves.

13.4 Rigid Body Dynamics

In a game engine, we are particularly concerned with the *kinematics* of objects—how they move over time. Many game engines include a *physics system* for the purposes of simulating the motion of the objects in the virtual game world in a somewhat physically realistic way. Technically speaking, game physics engines are typically concerned with a particular field of physics known as *dynamics*. This is the study of how *forces* affect the movement of objects. Until very recently, game physics systems have been focused almost exclusively on a specific subdiscipline known as *classical rigid body dynamics*. This name implies that in a game's physics simulation, two important simplifying assumptions are made:

- *Classical (Newtonian) mechanics*. The objects in the simulation are assumed to obey Newton's laws of motion. The objects are large enough that there are no quantum effects, and their speeds are low enough that there are no relativistic effects.
- *Rigid bodies*. All objects in the simulation are perfectly solid and cannot be deformed. In other words, their shape is constant. This idea meshes well with the assumptions made by the collision detection system. Furthermore, the assumption of rigidity greatly simplifies the mathematics required to simulate the dynamics of solid objects.

Game physics engines are also capable of ensuring that the motions of the rigid bodies in the game world conform to various *constraints*. The most common constraint is that of non-penetration—in other words, objects aren't allowed to pass through one another. Hence the physics system attempts to provide realistic *collision responses* whenever bodies are found to be interpen-

etrating.² This is one of the primary reasons for the tight interconnection between the physics engine and the collision detection system.

Most physics systems also allow game developers to set up other kinds of constraints in order to define realistic interactions between physically simulated rigid bodies. These may include hinges, prismatic joints (sliders), ball joints, wheels, “rag dolls” to emulate unconscious or dead characters and so on.

The physics system usually shares the collision world data structure, and in fact it usually drives the execution of the collision detection algorithm as part of its time step update routine. There is typically a one-to-one mapping between the rigid bodies in the dynamics simulation and the collidables managed by the collision engine. For example, in Havok, an `hkpRigidBody` object maintains a reference to one and only one `hkpCollidable` (although it is possible to create a collidable that has no rigid body). In PhysX, the two concepts are a bit more tightly integrated—an `NxActor` serves both as a collidable object and as a rigid body for the purposes of the dynamics simulation. These rigid bodies and their corresponding collidables are usually maintained in a singleton data structure known as the *collision/physics world*, or sometimes just the *physics world*.

The rigid bodies in the physics engine are typically distinct from the logical objects that make up the virtual world from a gameplay perspective. The positions and orientations of game objects can be driven by the physics simulation. To accomplish this, we query the physics engine every frame for the transform of each rigid body, and apply it in some way to the transform of the corresponding game object. It’s also possible for a game object’s motion, as determined by some other engine system (such as the animation system or the character control system) to drive the position and rotation of a rigid body in the physics world. As mentioned in Section 13.3.1, a single logical game object may be represented by one rigid body in the physics world, or by many. A simple object like a rock, weapon or barrel, might correspond to one rigid body. But an articulated character or a complex machine might be composed of many interconnected rigid pieces.

The remainder of this chapter will be devoted to investigating how game physics engines work. We’ll briefly introduce the theory that underlies rigid body dynamics simulations. Then we’ll investigate some of the most common features of a game physics system and have a look at how a physics engine might be integrated into a game.

²Or in the case of continuous collision detection, the collision response actually *prevents* the penetration from occurring.

13.4.1 Some Foundations

A great many excellent books, articles and slide presentations have been written on the topic of classical rigid body dynamics. A solid foundation in analytical mechanics theory can be obtained from [17]. Even more relevant to our discussion are texts like [39], [13] and [29], which have been written specifically about the kind of physics simulations done by games. Other texts, like [2], [11] and [32], include chapters on rigid body dynamics for games. Chris Hecker wrote a series of helpful articles on the topic of game physics for *Game Developer Magazine*; Chris has posted these and a variety of other useful resources at http://chrishecker.com/Rigid_Body_Dynamics. An informative slide presentation on dynamics simulation for games was produced by Russell Smith, the primary author of ODE; it is available at <http://www.ode.org/slides/parc/dynamics.pdf>.

In this section, I'll summarize the fundamental theoretical concepts that underlie the majority of game physics engines. This will be a whirlwind tour only, and by necessity I'll have to omit some details. Once you've read this chapter, I strongly encourage you to read at least a few of the additional resources cited previously.

13.4.1.1 Units

Most rigid body dynamics simulations operate in the MKS system of units. In this system, distance is measured in meters (abbreviated "m"), mass is measured in kilograms (abbreviated "kg") and time is measured in seconds (abbreviated "s"). Hence the name MKS.

You could configure your physics system to use other units if you wanted to, but if you do this, you need to make sure everything in the simulation is consistent. For example, constants like the acceleration due to gravity $\mathbf{g}$, which is measured in m/s^2 in the MKS system, would have to be re-expressed in whatever unit system you select. Most game teams just stick with MKS to keep life simple.

13.4.1.2 Separability of Linear and Angular Dynamics

An *unconstrained* rigid body is one that can translate freely along all three Cartesian axes and that can rotate freely about these three axes as well. We say that such a body has six *degrees of freedom* (DOF).

It is perhaps somewhat surprising that the motion of an unconstrained rigid body can be separated into two independent components:

- *Linear dynamics*. This is a description of the motion of the body when we ignore all rotational effects. (We can use linear dynamics alone to

describe the motion of an idealized *point mass*—i.e., a mass that is infinitesimally small and cannot rotate.)

- *Angular dynamics.* This is a description of the rotational motion of the body.

As you can well imagine, this ability to separate the linear and angular components of a rigid body's motion is extremely helpful when analyzing or simulating its behavior. It means that we can calculate a body's linear motion without regard to rotation—as if it were an idealized point mass—and then layer its angular motion on top in order to arrive at a complete description of the body's motion.

13.4.1.3 Center of Mass

For the purposes of *linear* dynamics, an unconstrained rigid body acts as though all of its mass were concentrated at a single point known as the *center of mass* (abbreviated CM, or sometimes COM). The center of mass is essentially the balancing point of the body for all possible orientations. In other words, the mass of a rigid body is distributed evenly around its center of mass in all directions.

For a body with uniform density, the center of mass lies at the *centroid* of the body. That is, if we were to divide the body up into N very small pieces, add up the positions of all these pieces as a vector sum and then divide by the number of pieces, we'd end up with a pretty good approximation to the location of the center of mass. If the body's density is not uniform, the position of each little piece would need to be *weighted* by that piece's mass, meaning that in general the center of mass is really a *weighted average* of the pieces' positions. So we have

$$\mathbf{r}_{\text{CM}} = \frac{\sum_{\forall i} m_i \mathbf{r}_i}{\sum_{\forall i} m_i} = \frac{\sum_{\forall i} m_i \mathbf{r}_i}{m},$$

where the symbol m represents the total mass of the body, and the symbol $\mathbf{r}$ represents a *radius vector* or *position vector*—i.e., a vector extending from the world-space origin to the point in question. (These sums become integrals in the limit as the sizes and masses of the little pieces approach zero.)

The center of mass always lies inside a convex body, although it may actually lie outside the body if it is concave. (For example, where would the center of mass of the letter “C” lie?)

13.4.2 Linear Dynamics

For the purposes of linear dynamics, the position of a rigid body can be fully described by a position vector $\mathbf{r}_{\text{CM}}$ that extends from the world-space origin to the center of mass of the body, as shown in Figure 13.22. Since we're using the MKS system, position is measured in meters (m). For the remainder of this discussion, we'll drop the CM subscripts, as it is understood that we are describing the motion of the body's center of mass.

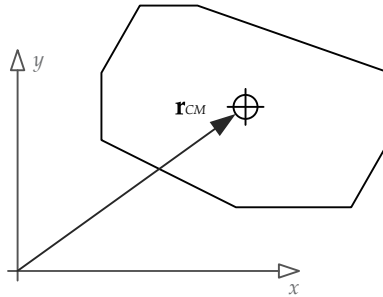


Figure 13.22. For the purposes of linear dynamics, the position of a rigid body can be fully described by the position of its center of mass.

13.4.2.1 Linear Velocity and Acceleration

The *linear velocity* of a rigid body defines the speed and direction in which the body's CM is moving. It is a vector quantity, typically measured in meters per second (m/s). Velocity is the first time derivative of position, so we can write

$$\mathbf{v}(t) = \frac{d\mathbf{r}(t)}{dt} = \dot{\mathbf{r}}(t),$$

where the dot over the vector $\mathbf{r}$ denotes taking the derivative with respect to time. Differentiating a vector is the same as differentiating each component independently, so

$$v_x(t) = \frac{dr_x(t)}{dt} = \dot{r}_x(t),$$

and so on for the y - and z -components.

Linear acceleration is the first derivative of linear velocity with respect to time, or the second derivative of the position of a body's CM versus time. Acceleration is a vector quantity, usually denoted by the symbol $\mathbf{a}$. So we can

write

$$\begin{aligned}\mathbf{a}(t) &= \frac{d\mathbf{v}(t)}{dt} = \dot{\mathbf{v}}(t) \\ &= \frac{d^2\mathbf{r}(t)}{dt^2} = \ddot{\mathbf{r}}(t).\end{aligned}$$

13.4.2.2 Force and Momentum

A *force* is defined as anything that causes an object with mass to accelerate or decelerate. A force has both a magnitude and a direction in space, so all forces are represented by vectors. A force is often denoted by the symbol $\mathbf{F}$. When N forces are applied to a rigid body, their net effect on the body's linear motion is found by simply adding up the force vectors:

$$\mathbf{F}_{\text{net}} = \sum_{i=1}^N \mathbf{F}_i.$$

Newton's famous Second Law states that force is proportional to acceleration and mass:

$$\mathbf{F}(t) = m\mathbf{a}(t) = m\ddot{\mathbf{r}}(t). \quad (13.2)$$

As Newton's law implies, force is measured in units of kilogram-meters per second squared ($\text{kg}\cdot\text{m}/\text{s}^2$). This unit is also called the Newton.

When we multiply a body's linear velocity by its mass, the result is a quantity known as *linear momentum*. It is customary to denote linear momentum with the symbol $\mathbf{p}$:

$$\mathbf{p}(t) = m\mathbf{v}(t).$$

When mass is constant, Equation (13.2) holds true. But if mass is not constant, as would be the case for a rocket whose fuel is being gradually used up and converted into energy, Equation (13.2) is not exactly correct. The proper formulation is actually as follows:

$$\mathbf{F}(t) = \frac{d\mathbf{p}(t)}{dt} = \frac{d(m(t)\mathbf{v}(t))}{dt}.$$

which of course reduces to the more familiar $\mathbf{F} = m\mathbf{a}$ when the mass is constant and can be brought outside the derivative. Linear momentum is not of much concern to us. However, the concept of momentum will become relevant when we discuss angular dynamics.

13.4.3 Solving the Equations of Motion

The central problem in rigid body dynamics is to solve for the motion of the body, given a set of known forces acting on it. For linear dynamics, this means finding $\mathbf{v}(t)$ and $\mathbf{r}(t)$ given knowledge of the net force $\mathbf{F}_{\text{net}}(t)$ and possibly other information, such as the position and velocity at some previous time. As we'll see below, this amounts to solving a pair of ordinary differential equations—one to find $\mathbf{v}(t)$ given $\mathbf{a}(t)$ and the other to find $\mathbf{r}(t)$ given $\mathbf{v}(t)$.

13.4.3.1 Force as a Function

A force can be constant, or it can be a function of time as shown above. A force can also be a function of the position of the body, its velocity, or any number of other quantities. So in general, the expression for force should really be written as follows:

$$\mathbf{F}(t, \mathbf{r}(t), \mathbf{v}(t), \dots) = m\mathbf{a}(t). \quad (13.3)$$

This can be rewritten in terms of the position vector and its first and second derivatives as follows:

$$\mathbf{F}(t, \mathbf{r}(t), \dot{\mathbf{r}}(t), \dots) = m\ddot{\mathbf{r}}(t).$$

For example, the force exerted by a spring is proportional to how far it has been stretched away from its natural resting position. In one dimension, with the spring's resting position at $x = 0$, we can write

$$F(t, x(t)) = -kx(t),$$

where k is the *spring constant*, a measure of the spring's stiffness.

As another example, the damping force exerted by a mechanical viscous damper (a so-called dashpot) is proportional to the velocity of the damper's piston. So in one dimension, we can write

$$F(t, v(t)) = -bv(t),$$

where b is a *viscous damping coefficient*.

13.4.3.2 Ordinary Differential Equations

In general, an *ordinary differential equation* (ODE) is an equation involving a function of one independent variable and various derivatives of that function. If our independent variable is time and our function is $x(t)$, then an ODE is a relation of the form

$$\frac{d^n x}{dt^n} = f\left(t, x(t), \frac{dx(t)}{dt}, \frac{d^2 x(t)}{dt^2}, \dots, \frac{d^{n-1} x(t)}{dt^{n-1}}\right).$$

Put another way, the n th derivative of $x(t)$ is expressed as a function f whose arguments can be time (t), position ($x(t)$), and any number of derivatives of $x(t)$ as long as those derivatives are of *lower order* than n .

As we saw in Equation (13.3), force is a function of time, position and velocity in general:

$$\ddot{\mathbf{r}}(t) = \frac{1}{m} \mathbf{F}(t, \mathbf{r}(t), \dot{\mathbf{r}}(t)).$$

This clearly qualifies as an ODE. We wish to solve this ODE in order to find $\mathbf{v}(t)$ and $\mathbf{r}(t)$.

13.4.3.3 Analytical Solutions

In some rare situations, the differential equations of motion can be solved *analytically*, meaning that a simple, closed-form function can be found that describes the body's position for *all possible values* of time t . A common example is the vertical motion of a projectile under the influence of a constant acceleration due to gravity, $\mathbf{a}(t) = [0, g, 0]$, where $g = -9.8\text{m/s}^2$. In this case, the ODE of motion boils down to

$$\ddot{y}(t) = g.$$

Integrating once yields

$$\dot{y}(t) = gt + v_0,$$

where v_0 is the vertical velocity at time $t = 0$. Integrating a second time yields the familiar solution

$$y(t) = \frac{1}{2}gt^2 + v_0t + y_0,$$

where y_0 is the initial vertical position of the object.

However, analytical solutions are almost never possible in game physics. This is due in part to the fact that closed-form solutions to some differential equations are simply not known. Moreover, a game is an interactive simulation, so we usually cannot predict how the forces in a game will behave over time. This makes it impossible to find simple, closed-form expressions for the positions and velocities of the objects in the game as functions of time.

There are of course exceptions to this rule of thumb. For example, it's pretty common to solve for a closed-form expression in order to determine with what velocity a projectile must be launched in order to hit a predefined target.

13.4.4 Numerical Integration

For the reasons cited above, game physics engines turn to a technique known as *numerical integration*. With this technique, we solve our differential equations in a *time-stepped* manner—using the solution from a previous time step

to arrive at the solution for the next time step. The duration of the time step is usually taken to be (roughly) constant and is denoted by the symbol Δt . Given that we know the body's position and velocity at the current time t_1 and that the force is known as a function of time, position and/or velocity, we wish to find the position and velocity at the next time step $t_2 = t_1 + \Delta t$. In other words, given $\mathbf{r}(t_1)$, $\mathbf{v}(t_1)$ and $\mathbf{F}(t, \mathbf{r}, \mathbf{v})$, the problem is to find $\mathbf{r}(t_2)$ and $\mathbf{v}(t_2)$.

13.4.4.1 Explicit Euler

One of the simplest numerical solutions to an ODE is known as the *explicit Euler method*. This is the intuitive approach often taken by new game programmers. Let's assume for the moment that we already know the current velocity and that we wish to solve the following ODE to find the body's position on the next frame:

$$\mathbf{v}(t) = \dot{\mathbf{r}}(t). \quad (13.4)$$

Using the explicit Euler method, we simply convert the velocity from meters per second into meters per frame by multiplying by the time delta, and then we add "one frame's worth" of velocity onto the current position in order to find the new position on the next frame. This yields the following approximate solution to the ODE given by Equation (13.4):

$$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \mathbf{v}(t_1)\Delta t. \quad (13.5)$$

We can take an analogous approach to find the body's velocity next frame given the net force acting this frame. Hence, the approximate explicit Euler solution to the ODE

$$\mathbf{a}(t) = \frac{\mathbf{F}_{\text{net}}(t)}{m} = \dot{\mathbf{v}}(t)$$

is as follows:

$$\mathbf{v}(t_2) = \mathbf{v}(t_1) + \frac{\mathbf{F}_{\text{net}}(t)}{m}\Delta t. \quad (13.6)$$

Interpretations of Explicit Euler

What we're really doing in Equation (13.5) is assuming that the velocity of the body is constant during the time step. Therefore, we can use the *current* velocity to predict the body's position on the *next* frame. The change in position $\Delta \mathbf{r}$ between times t_1 and t_2 is hence $\Delta \mathbf{r} = \mathbf{v}(t_1)\Delta t$. Graphically, if we imagine a plot of the position of the body versus time, we are taking the *slope* of the function at time t_1 (which is just $\mathbf{v}(t_1)$) and extrapolating it linearly to the next time step t_2 . As we can see in Figure 13.23, linear extrapolation does not necessarily provide us with a particularly good estimate of the true position at the

next time step $\mathbf{r}(t_2)$, but it does work reasonably well as long as the velocity is roughly constant.

Figure 13.23 suggests another way to interpret the explicit Euler method—as an approximation of a derivative. By definition, any derivative is the quotient of two infinitesimally small differences (in our case, $d\mathbf{r}/dt$). The explicit Euler method approximates this using the quotient of two *finite differences*. In other words, $d\mathbf{r}$ becomes $\Delta\mathbf{r}$ and dt becomes Δt . This yields

$$\frac{d\mathbf{r}}{dt} \approx \frac{\Delta\mathbf{r}}{\Delta t};$$

$$\mathbf{v}(t_1) \approx \frac{\mathbf{r}(t_2) - \mathbf{r}(t_1)}{t_2 - t_1},$$

which again simplifies to Equation (13.5). This approximation is really only valid when the velocity is constant over the time step. It is also valid in the limit as Δt tends toward zero (at which point it becomes *exactly* right). Obviously, this same analysis can be applied to Equation (13.6) as well.

13.4.4.2 Properties of Numerical Methods

We’ve implied that the explicit Euler method is not particularly accurate. Let’s pin this idea down more concretely. A numerical solution to an ordinary differential equation actually has three important and interrelated properties:

- *Convergence.* As the time step Δt tends toward zero, does the approximate solution get closer and closer to the real solution?
- *Order.* Given a particular numerical approximation to the solution of an ODE, how “bad” is the error? Errors in numerical ODE solutions are typically proportional to some power of the time step duration Δt , so they are often written using “big O” notation (e.g., $O(\Delta t^2)$). We say

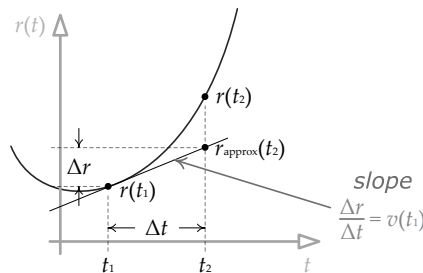


Figure 13.23. In the explicit Euler method, the slope of $\mathbf{r}(t)$ at time t_1 is used to linearly extrapolate from $\mathbf{r}(t_1)$ to an estimate of the true value of $\mathbf{r}(t_2)$.

that a particular numerical method is of “order n ” when its error term is $O(\Delta t^{(n+1)})$.

- *Stability.* Does the numerical solution tend to “settle down” over time? If a numerical method adds energy into the system, object velocities will eventually “explode,” and the system will become *unstable*. On the other hand, if a numerical method tends to remove energy from the system, it will have an overall damping effect, and the system will be *stable*.

The concept of order warrants a little more explanation. We usually measure the error of a numerical method by comparing its approximate equation with the infinite Taylor series expansion of the exact solution to the ODE. We then cancel terms by subtracting the two equations. The remaining Taylor terms represent the error inherent in the method. For example, the explicit Euler equation is

$$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \dot{\mathbf{r}}(t_1)\Delta t.$$

The infinite Taylor series expansion of the exact solution is

$$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \dot{\mathbf{r}}(t_1)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t_1)\Delta t^2 + \frac{1}{6}\mathbf{r}^{(3)}(t_1)\Delta t^3 + \dots,$$

where $\mathbf{r}^{(3)}$ represents the third derivative with respect to time. Therefore, the error is represented by all of the terms after the $\mathbf{v}\Delta t$ term, which is of order $O(\Delta t^2)$ (because this term dwarfs the other higher-order terms):

$$\begin{aligned} \mathbf{E} &= \frac{1}{2}\ddot{\mathbf{r}}(t_1)\Delta t^2 + \frac{1}{6}\mathbf{r}^{(3)}(t_1)\Delta t^3 + \dots \\ &= O(\Delta t^2). \end{aligned}$$

To make the error of a method explicit, we’ll often write its equation with the error term added in “big O” notation at the end. For example, the explicit Euler method’s equation is most accurately written as follows:

$$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \dot{\mathbf{r}}(t_1)\Delta t + O(\Delta t^2).$$

We say that the explicit Euler method is a “first-order” method because it is accurate up to *and including* the Taylor series term involving Δt to the first power. In general, if a method’s error term is $O(\Delta t^{(n+1)})$, then it is said to be an “order n ” method.

13.4.4.3 Alternatives to Explicit Euler

The explicit Euler method sees quite a lot of use for simple integration tasks in games, producing the best results when the velocity is nearly constant. However, it is not used in general-purpose dynamics simulations because of its

high error and poor stability. There are all sorts of other numerical methods for solving ODEs, including backward Euler (another first-order method), midpoint Euler (a second-order method) and the family of Runge-Kutta methods. (The fourth-order Runge-Kutta, often abbreviated “RK4,” is particularly popular.) We won’t describe these in any detail here, as you can find voluminous amounts of information about them online and in the literature. The Wikipedia page http://en.wikipedia.org/wiki/Numerical_ordinary_differential_equations serves as an excellent jumping-off point for learning these methods.

13.4.4.4 Verlet Integration

The numerical ODE method most often used in interactive games these days is probably the Verlet method, so I’ll take a moment to describe it in some detail. There are actually two variants of this method: regular Verlet and the so-called *velocity Verlet*. I’ll present both methods here, but I’ll leave the theory and deep explanations to the myriad papers and Web pages available on the topic. (For a start, check out http://en.wikipedia.org/wiki/Verlet_integration.)

The regular Verlet method is attractive because it achieves a high order (low error), is relatively simple and inexpensive to evaluate, and produces a solution for position directly in terms of acceleration in one step (as opposed to the two steps normally required to go from acceleration to velocity and then from velocity to position). The formula is derived by adding two Taylor series expansions, one going forward in time and one going backward in time:

$$\mathbf{r}(t_1 + \Delta t) = \mathbf{r}(t_1) + \dot{\mathbf{r}}(t_1)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t_1)\Delta t^2 + \frac{1}{6}\mathbf{r}^{(3)}(t_1)\Delta t^3 + O(\Delta t^4);$$

$$\mathbf{r}(t_1 - \Delta t) = \mathbf{r}(t_1) - \dot{\mathbf{r}}(t_1)\Delta t + \frac{1}{2}\ddot{\mathbf{r}}(t_1)\Delta t^2 - \frac{1}{6}\mathbf{r}^{(3)}(t_1)\Delta t^3 + O(\Delta t^4).$$

Adding these expressions causes the negative terms to cancel with the corresponding positive ones. The result gives us the position at the next time step in terms of the acceleration and the two (known) positions at the current and previous time steps. This is the regular Verlet method:

$$\mathbf{r}(t_1 + \Delta t) = 2\mathbf{r}(t_1) - \mathbf{r}(t_1 - \Delta t) + \mathbf{a}(t_1)\Delta t^2 + O(\Delta t^4).$$

In terms of net force, the Verlet method becomes

$$\mathbf{r}(t_1 + \Delta t) = 2\mathbf{r}(t_1) - \mathbf{r}(t_1 - \Delta t) + \frac{\mathbf{F}_{\text{net}}(t_1)}{m}\Delta t^2 + O(\Delta t^4).$$

The velocity is conspicuously absent from this expression. However, it can be found using the following somewhat inaccurate approximation (among other alternatives):

$$\mathbf{v}(t_1 + \Delta t) = \frac{\mathbf{r}(t_1 + \Delta t) - \mathbf{r}(t_1)}{\Delta t} + O(\Delta t).$$

13.4.4.5 Velocity Verlet

The more commonly used *velocity Verlet* method is a four-step process in which the time step is divided into two parts to facilitate the solution. Given that $\mathbf{a}(t_1) = \frac{1}{m}\mathbf{F}(t_1, \mathbf{r}(t_1), \mathbf{v}(t_1))$ is known, we do the following:

1. Calculate $\mathbf{r}(t_1 + \Delta t) = \mathbf{r}(t_1) + \mathbf{v}(t_1)\Delta t + \frac{1}{2}\mathbf{a}(t_1)\Delta t^2$.
2. Calculate $\mathbf{v}(t_1 + \frac{1}{2}\Delta t) = \mathbf{v}(t_1) + \frac{1}{2}\mathbf{a}(t_1)\Delta t$.
3. Determine $\mathbf{a}(t_1 + \Delta t) = \mathbf{a}(t_2) = \frac{1}{m}\mathbf{F}(t_2, \mathbf{r}(t_2), \mathbf{v}(t_2))$.
4. Calculate $\mathbf{v}(t_1 + \Delta t) = \mathbf{v}(t_1 + \frac{1}{2}\Delta t) + \frac{1}{2}\mathbf{a}(t_1 + \Delta t)\Delta t$.

Notice in the third step that the force function depends on the position and velocity on the *next* time step, $\mathbf{r}(t_2)$ and $\mathbf{v}(t_2)$. We already calculated $\mathbf{r}(t_2)$ in step 1, so we have all the information we need as long as the force is not velocity-dependent. If it is velocity-dependent, then we must approximate the next frame's velocity, perhaps using the explicit Euler method.

13.4.5 Angular Dynamics in Two Dimensions

Up until now, we've focused on analyzing the linear motion of a body's center of mass (which acts as if it were a point mass). As I said earlier, an unconstrained rigid body will rotate about its center of mass. This means that we can layer the angular motion of a body on top of the linear motion of its center of mass in order to arrive at a complete description of the body's overall motion. The study of a body's rotational motion in response to applied forces is called *angular dynamics*.

In two dimensions, angular dynamics works almost identically to linear dynamics. For each linear quantity, there's an angular analog, and the mathematics works out quite neatly. So let's investigate two-dimensional angular dynamics first. As we'll see, when we extend the discussion into three dimensions, things get a bit messier, but we'll burn that bridge when we get to it!

13.4.5.1 Orientation and Angular Speed

In two dimensions, every rigid body can be treated as a thin sheet of material. (Some physics texts refer to such a body as a *plane lamina*.) All linear motion occurs in the xy -plane, and all rotations occur about the z -axis. (Visualize wooden puzzle pieces sliding about on an air hockey table.)

The orientation of a rigid body in 2D is fully described by an angle θ , measured in radians relative to some agreed-upon zero rotation. For example, we

might specify that $\theta = 0$ when a race car is facing directly down the positive x -axis in world space. This angle is of course a time-varying function, so we denote it $\theta(t)$.

13.4.5.2 Angular Speed and Acceleration

Angular velocity measures the rate at which a body's rotation angle changes over time. In two dimensions, angular velocity is a scalar, more correctly called angular *speed*, since the term "velocity" really only applies to vectors. It is denoted by the scalar function $\omega(t)$ and measured in radians per second (rad/s). Angular speed is the derivative of the orientation angle $\theta(t)$ with respect to time:

Angular:	Linear:
$\omega(t) = \frac{d\theta(t)}{dt} = \dot{\theta}(t)$	$\mathbf{v}(t) = \frac{d\mathbf{r}(t)}{dt} = \dot{\mathbf{r}}(t).$

And as we'd expect, angular acceleration, denoted $\alpha(t)$ and measured in radians per second squared (rad/s²), is the rate of change of angular speed:

Angular:	Linear:
$\alpha(t) = \frac{d\omega(t)}{dt} = \dot{\omega}(t) = \ddot{\theta}(t)$	$\mathbf{a}(t) = \frac{d\mathbf{v}(t)}{dt} = \dot{\mathbf{v}}(t) = \ddot{\mathbf{r}}(t).$

13.4.5.3 Moment of Inertia

The rotational equivalent of mass is a quantity known as the *moment of inertia*. Just as mass describes how easy or difficult it is to change the linear velocity of a point mass, the moment of inertia measures how easy or difficult it is to change the angular speed of a rigid body about a particular axis. If a body's mass is concentrated near an axis of rotation, it will be relatively easier to rotate about that axis, and it will hence have a smaller moment of inertia than a body whose mass is spread out away from that axis.

Since we're focusing on two-dimensional angular dynamics right now, the axis of rotation is always z , and a body's moment of inertia is a simple scalar value. Moment of inertia is usually denoted by the symbol I . We won't get into the details of how to calculate the moment of inertia here. For a full derivation, see [17].

13.4.5.4 Torque

Until now, we've assumed that all forces are applied to the center of mass of a rigid body. However, in general, forces can be applied at arbitrary points on a

body. If the line of action of a force passes through the body's center of mass, then the force will produce linear motion only, as we've already seen. Otherwise, the force will introduce a rotational force known as a *torque* in addition to the linear motion it normally causes. This is illustrated in Figure 13.24.

We can calculate torque using a cross product. First, we express the location at which the force is applied as a vector $\mathbf{r}$ extending from the body's center of mass to the point of application of the force. (In other words, the vector $\mathbf{r}$ is in *body space*, where the origin of body space is defined to be the center of mass.) This is illustrated in Figure 13.25. The torque $\mathbf{N}$ caused by a force $\mathbf{F}$ applied at a location $\mathbf{r}$ is

$$\mathbf{N} = \mathbf{r} \times \mathbf{F}. \quad (13.7)$$

Equation (13.7) implies that torque increases as the force is applied farther from the center of mass. This explains why a lever can help us to move a heavy object. It also explains why a force applied directly through the center of mass produces no torque and no rotation—the magnitude of the vector $\mathbf{r}$ is zero in this case.

When two or more forces are applied to a rigid body, the torque vectors produced by each one can be summed, just as we can sum forces. So in general we are interested in the net torque, $\mathbf{N}_{\text{net}}$.

In two dimensions, the vectors $\mathbf{r}$ and $\mathbf{F}$ must both lie in the xy -plane, so $\mathbf{N}$ will always be directed along the positive or negative z -axis. As such, we'll denote a two-dimensional torque via the scalar N_z , which is just the z -component of the vector $\mathbf{N}$.

Torque is related to angular acceleration and moment of inertia in much

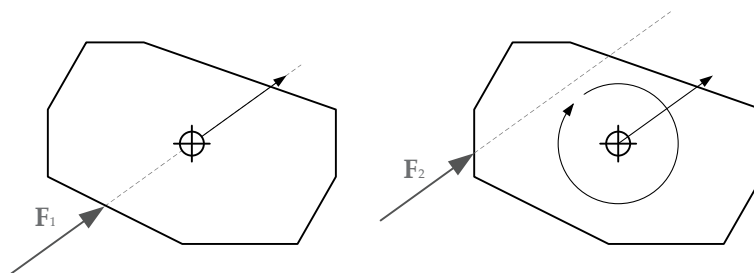


Figure 13.24. On the left, a force applied to a body's CM produces purely linear motion. On the right, a force applied off-center will give rise to a torque, producing rotational motion as well as linear motion.

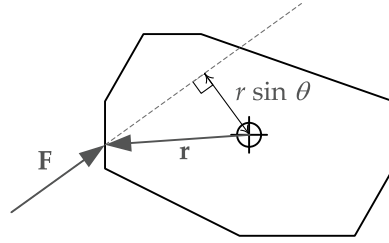


Figure 13.25. Torque is calculated by taking the cross product between a force's point of application in body space (i.e., relative to the center of mass) and the force vector. The vectors are shown here in two dimensions for ease of illustration; if it could be drawn, the torque vector would be directed into the page.

the same way that force is related to linear acceleration and mass:

Angular:		Linear:	
$N_z(t) = I\alpha(t) = I\dot{\omega}(t) = I\ddot{\theta}(t)$		$\mathbf{F}(t) = m\mathbf{a}(t) = m\dot{\mathbf{v}}(t) = m\ddot{\mathbf{r}}(t).$	(13.8)

13.4.5.5 Solving the Angular Equations of Motion in Two Dimensions

For the two-dimensional case, we can solve the angular equations of motion using exactly the same numerical integration techniques we applied to the linear dynamics problem. The pair of ODEs that we wish to solve is as follows:

Angular:		Linear:
$N_{\text{net}}(t) = I\dot{\omega}(t)$		$\mathbf{F}_{\text{net}}(t) = m\dot{\mathbf{v}}(t)$
$\omega(t) = \dot{\theta}(t)$		$\mathbf{v}(t) = \dot{\mathbf{r}}(t),$

and their approximate explicit Euler solutions are

Angular:		Linear:
$\omega(t_2) = \omega(t_1) + I^{-1}N_{\text{net}}(t_1)\Delta t$		$\mathbf{v}(t_2) = \mathbf{v}(t_1) + m^{-1}\mathbf{F}_{\text{net}}(t_1)\Delta t$
$\theta(t_2) = \theta(t_1) + \omega(t_1)\Delta t$		$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \mathbf{v}(t_1)\Delta t.$

Of course, we could apply any of the other more-accurate numerical methods as well, such as the velocity Verlet method (I've omitted the linear case here for compactness, but compare this to the steps given in Section 13.4.4.5):

1. Calculate $\theta(t_1 + \Delta t) = \theta(t_1) + \omega(t_1)\Delta t + \frac{1}{2}\alpha(t_1)\Delta t^2$.
2. Calculate $\omega(t_1 + \frac{1}{2}\Delta t) = \omega(t_1) + \frac{1}{2}\alpha(t_1)\Delta t$.

3. Calculate $\alpha(t_1 + \Delta t) = \alpha(t_2) = I^{-1}N_{\text{net}}(t_2, \theta(t_2), \omega(t_2))$.
4. Calculate $\omega(t_1 + \Delta t) = \omega(t_1 + \frac{1}{2}\Delta t) + \frac{1}{2}\alpha(t_1 + \Delta t)\Delta t$.

13.4.6 Angular Dynamics in Three Dimensions

Angular dynamics in three dimensions is a somewhat more complex topic than its two-dimensional counterpart, although the basic concepts are of course very similar. In the following section, I'll give a very brief overview of how angular dynamics works in 3D, focusing primarily on the things that are typically confusing to someone who is new to the topic. For further information, check out Glenn Fiedler's series of articles on the topic, available at <http://gafferongames.com/game-physics/physics-in-3d/>. Another helpful resource is the paper entitled "An Introduction to Physically Based Modeling" by David Baraff of the Robotics Institute at Carnegie Mellon University, available at <http://www-2.cs.cmu.edu/~baraff/sigcourse/notesd1.pdf>.

13.4.6.1 The Inertia Tensor

A rigid body may have a very different distribution of mass about the three coordinate axes. As such, we should expect a body to have different moments of inertia about different axes. For example, a long thin rod should be relatively easy to make rotate about its long axis because all the mass is concentrated very close to the axis of rotation. Likewise, the rod should be relatively more difficult to make rotate about its short axis because its mass is spread out farther from the axis. This is indeed the case, and it is why a figure skater spins faster when she tucks her limbs in close to her body.

In three dimensions, the rotational mass of a rigid body is represented by a 3×3 matrix known as its *inertia tensor*. It is usually represented by the symbol $\mathbf{I}$ (as before, we won't describe how to calculate the inertia tensor here; see [17] for details):

$$\mathbf{I} = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix}.$$

The elements lying along the diagonal of this matrix are the moments of inertia of the body about its three principal axes, I_{xx} , I_{yy} and I_{zz} . The off-diagonal elements are called *products of inertia*. They are zero when the body is symmetrical about all three principal axes (as would be the case for a rectangular box). When they are nonzero, they tend to produce physically realistic yet somewhat unintuitive motions that the average game player would probably think were "wrong" anyway. Therefore, the inertia tensor is often

simplified down to the three-element vector $[I_{xx} \ I_{yy} \ I_{zz}]$ in game physics engines.

13.4.6.2 Orientation in Three Dimensions

In two dimensions, we know that the orientation of a rigid body can be described by a single angle θ , which measures rotation about the z -axis (assuming the motion is taking place in the xy -plane). In three dimensions, a body's orientation could be represented using three Euler angles $[\theta_x \ \theta_y \ \theta_z]$, each representing the body's rotation about one of the three Cartesian axes. However, as we saw in Chapter 5, Euler angles suffer from gimbal lock problems and can be difficult to work with mathematically. Therefore, the orientation of a body is more often represented using either a 3×3 matrix $\mathbf{R}$ or a unit quaternion $\mathbf{q}$. We'll use the quaternion form exclusively in this chapter.

Recall that a quaternion is a four-element vector whose x -, y - and z -components can be interpreted as a unit vector $\mathbf{u}$ lying along the axis of rotation, scaled by the sine of the half-angle and whose w component is the cosine of the half-angle:

$$\begin{aligned} \mathbf{q} &= [q_x \ q_y \ q_z \ q_w] \\ &= [\mathbf{q} \ q_w] \\ &= [\mathbf{u} \sin \frac{\theta}{2} \ \cos \frac{\theta}{2}]. \end{aligned}$$

A body's orientation is of course a function of time, so we should write it $\mathbf{q}(t)$.

Again, we need to select an arbitrary direction to be our zero rotation. For example, we might say that by default, the front of every object will lie along the positive z -axis in world space, with y up and x to the left. Any non-identity quaternion will serve to rotate the object away from this canonical world-space orientation. The choice of the canonical orientation is arbitrary, but of course it's important to be consistent across all assets in the game.

13.4.6.3 Angular Velocity and Momentum in Three Dimensions

In three dimensions, angular velocity is a vector quantity, denoted by $\boldsymbol{\omega}(t)$. The angular velocity vector can be visualized as a unit-length vector $\mathbf{u}$ that defines the axis of rotation, scaled by the two-dimensional angular velocity $\omega_u = \dot{\theta}_u$ of the body about the u -axis. Hence,

$$\boldsymbol{\omega}(t) = \omega_u(t)\mathbf{u} = \dot{\theta}_u(t)\mathbf{u}.$$

In linear dynamics, we saw that if there are no forces acting on a body, then the linear acceleration is zero, and linear velocity is constant. In two-dimensional angular dynamics, this again holds true: If there are no torques

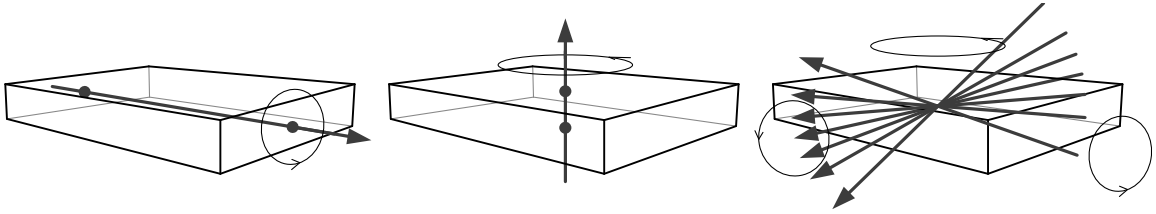


Figure 13.26. A rectangular object that is spun about its shortest or longest axis has a constant angular velocity vector. However, when spun about its medium-sized axis, the direction of the angular velocity vector changes wildly.

acting on a body in two dimensions, then the angular acceleration α is zero, and the angular speed ω about the z -axis is constant.

Unfortunately, this is *not* the case in three dimensions. It turns out that even when a rigid body is rotating in the absence of all forces, its angular velocity vector $\boldsymbol{\omega}(t)$ may not be constant because the axis of rotation can continually change direction. You can see this effect in action when you try to spin a rectangular object, like a block of wood, in mid-air in front of you. If you throw the block so that it is rotating about its shortest axis, it will spin in a stable way. The orientation of the axis stays roughly constant. The same thing happens if you try to spin the block about its longest axis. But if you try to spin the block around the remaining axis (the one that's neither the shortest nor the longest), the rotation will be utterly unstable. (Try it! Go steal a wooden block from a baby and spin it in various ways. On second thought, make sure to give it back when you're done.) The axis of rotation itself changes direction wildly as the object spins. This is illustrated in Figure 13.26.

The fact that the angular velocity vector can change in the absence of torques is another way of saying that angular velocity is not *conserved*. However, a related quantity called the *angular momentum* does remain constant in the absence of forces and hence *is* conserved. Angular momentum is the rotational equivalent of linear momentum:

Angular:	Linear:
$\mathbf{L}(t) = \mathbf{I}\boldsymbol{\omega}(t)$	$\mathbf{p}(t) = m\mathbf{v}(t).$

Like the linear case, angular momentum $\mathbf{L}(t)$ is a three-element vector. However, unlike the linear case, rotational mass (the inertia tensor) is not a scalar but rather a 3×3 matrix. As such, the expression $\mathbf{I}\boldsymbol{\omega}$ is computed via a

matrix multiplication:

$$\begin{bmatrix} L_x(t) \\ L_y(t) \\ L_z(t) \end{bmatrix} = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix} \begin{bmatrix} \omega_x(t) \\ \omega_y(t) \\ \omega_z(t) \end{bmatrix}.$$

Because the angular velocity $\boldsymbol{\omega}$ is not conserved, we do not treat it as a primary quantity in our dynamics simulations the way we do the linear velocity $\mathbf{v}$. Instead, we treat angular momentum $\mathbf{L}$ as the primary quantity. The angular velocity is a secondary quantity, determined only after we have determined the value of $\mathbf{L}$ at each time step of the simulation.

13.4.6.4 Torque in Three Dimensions

In three dimensions, we still calculate torque as the cross product between the radial position vector of the point of force application and the force vector itself ($\mathbf{N} = \mathbf{r} \times \mathbf{F}$). Equation (13.8) still holds, but we always write it in terms of the angular momentum because angular velocity is not a conserved quantity:

$$\begin{aligned} \mathbf{N} &= \mathbf{I}\boldsymbol{\alpha}(t) \\ &= \mathbf{I} \frac{d\boldsymbol{\omega}(t)}{dt} \\ &= \frac{d}{dt}(\mathbf{I}\boldsymbol{\omega}(t)) \\ &= \frac{d\mathbf{L}(t)}{dt}. \end{aligned}$$

13.4.6.5 Solving the Equations of Angular Motion in Three Dimensions

When solving the equations of angular motion in three dimensions, we might be tempted to take exactly the same approach we used for linear motion and two-dimensional angular motion. We might guess that the differential equations of motion should be written

Angular 3D?	Linear:
$\mathbf{N}_{\text{net}}(t) = \mathbf{I}\dot{\boldsymbol{\omega}}(t)$	$\mathbf{F}_{\text{net}}(t) = m\dot{\mathbf{v}}(t)$
$\boldsymbol{\omega}(t) = \dot{\boldsymbol{\theta}}(t)$	$\mathbf{v}(t) = \dot{\mathbf{r}}(t),$

and using the explicit Euler method, we might guess that the approximate solutions to these ODEs would look something like this:

Angular 3D?	Linear:
$\boldsymbol{\omega}(t_2) = \boldsymbol{\omega}(t_1) + \mathbf{I}^{-1}\mathbf{N}_{\text{net}}(t_1)\Delta t$	$\mathbf{v}(t_2) = \mathbf{v}(t_1) + m^{-1}\mathbf{F}_{\text{net}}(t)\Delta t$
$\boldsymbol{\theta}(t_2) = \boldsymbol{\theta}(t_1) + \boldsymbol{\omega}(t_1)\Delta t$	$\mathbf{r}(t_2) = \mathbf{r}(t_1) + \mathbf{v}(t_1)\Delta t.$

However, this is *not* actually correct. The differential equations of three-dimensional angular motion differ from their linear and two-dimensional angular counterparts in two important ways:

1. Instead of solving for the angular velocity $\boldsymbol{\omega}$, we solve for the angular momentum $\mathbf{L}$ directly. We then calculate the angular velocity vector as a secondary quantity using $\mathbf{I}$ and $\mathbf{L}$. We do this because angular momentum is conserved, while angular velocity is not.
2. When solving for the orientation given the angular velocity, we have a problem: The angular velocity is a three-element *vector*, while the orientation is a four-element *quaternion*. How can we write an ODE relating a quaternion to a vector? The answer is that we cannot, at least not directly. But what we can do is convert the angular velocity vector into quaternion form and then apply a slightly odd-looking equation that relates the orientation quaternion to the angular velocity quaternion.

It turns out that when we express a rigid body's orientation as a quaternion, the derivative of this quaternion is related to the body's angular velocity vector in the following way. First, we construct an *angular velocity quaternion*. This quaternion contains the three components of the angular velocity vector in x , y and z , with its w -component set to zero:

$$\boldsymbol{\omega} = [\omega_x \quad \omega_y \quad \omega_z \quad 0].$$

Now the differential equation relating the orientation quaternion to the angular velocity quaternion is (for reasons we won't get into here) as follows:

$$\frac{d\boldsymbol{\omega}(t)}{dt} = \dot{\mathbf{q}}(t) = \frac{1}{2}\boldsymbol{\omega}(t)\mathbf{q}(t).$$

It's important to remember here that $\boldsymbol{\omega}(t)$ is the angular velocity *quaternion* as described above and that the product $\boldsymbol{\omega}(t)\mathbf{q}(t)$ is a *quaternion* product (see Section 5.4.2.1 for details).

So, we actually need to write the ODEs of motion as follows (note that I've recast the linear ODEs in terms of linear momentum as well, to underscore the similarities between the two cases):

Angular 3D:	Linear:
$\mathbf{N}_{\text{net}}(t) = \dot{\mathbf{L}}(t)$	$\mathbf{F}_{\text{net}}(t) = \dot{\mathbf{p}}(t)$
$\boldsymbol{\omega}(t) = \mathbf{I}^{-1}\mathbf{L}(t)$	$\mathbf{v}(t) = m^{-1}\mathbf{p}(t)$
$\boldsymbol{\omega}(t) = [\boldsymbol{\omega}(t) \quad 0]$	$\mathbf{v}(t) = \dot{\mathbf{r}}(t).$
$\frac{1}{2}\boldsymbol{\omega}(t)\mathbf{q}(t) = \dot{\mathbf{q}}(t)$	

Using the explicit Euler method, the final approximate solution to the angular ODEs in three dimensions is actually as follows:

$$\begin{aligned}\mathbf{L}(t_2) &= \mathbf{L}(t_1) + \mathbf{N}_{\text{net}}(t_1)\Delta t && \text{(vectors)} \\ &= \mathbf{L}(t_1) + \Delta t \sum_{\forall i} (\mathbf{r}_i \times \mathbf{F}_i(t_1)); && \text{(vectors)} \\ \omega(t_2) &= [\mathbf{I}^{-1}\mathbf{L}(t_2) \quad 0]; && \text{(quaternions)} \\ \mathbf{q}(t_2) &= \mathbf{q}(t_1) + \frac{1}{2}\omega(t_1)\mathbf{q}(t_1)\Delta t. && \text{(quaternions)}\end{aligned}$$

The orientation quaternion $\mathbf{q}(t)$ should be renormalized periodically to reverse the effects of the inevitable accumulation of floating-point error.

As always, the explicit Euler method is being used here just as an example. In a real engine, we would employ velocity Verlet, RK4 or some other more-stable and more-accurate numerical method.

13.4.7 Collision Response

Everything we've discussed so far assumes that our rigid bodies are neither colliding with anything, nor is their motion constrained in any other way. When bodies collide with one another, the dynamics simulation must take steps to ensure that they respond realistically to the collision and that they are never left in a state of interpenetration after the simulation step has been completed. This is known as *collision response*.

13.4.7.1 Energy

Before we discuss collision response, we must understand the concept of *energy*. When a force moves a body over a distance, we say that the force does *work*. Work represents a change in energy—that is, a force either adds energy to a system of rigid bodies (e.g., an explosion) or it removes energy from the system (e.g., friction). Energy comes in two forms. The potential energy V of a body is the energy it has simply because of where it is relative to a force field such as a gravitational or a magnetic field. (For example, the higher up a body is above the surface of the Earth, the more gravitational potential energy it has.) The kinetic energy of a body T represents the energy arising from the fact that it is moving relative to other bodies in a system. The total energy $E = V + T$ of an isolated system of bodies is a *conserved* quantity, meaning that it remains constant unless energy is being drained from the system or added from outside the system.

The kinetic energy arising from linear motion can be written

$$T_{\text{linear}} = \frac{1}{2}mv^2,$$

or in terms of the linear momentum and velocity vectors:

$$T_{\text{linear}} = \frac{1}{2} \mathbf{p} \cdot \mathbf{v}.$$

Analogously, the kinetic energy arising from a body's rotational motion is as follows:

$$T_{\text{angular}} = \frac{1}{2} \mathbf{L} \cdot \boldsymbol{\omega}.$$

Energy and its conservation can be extremely useful concepts when solving all sorts of physics problems. We'll see the role that energy plays in the determination of collision responses in the following section.

13.4.7.2 Impulsive Collision Response

When two bodies collide in the real world, a complex set of events takes place. The bodies compress slightly and then rebound, changing their velocities and losing energy to sound and heat in the process. Most real-time rigid body dynamics simulations approximate all of these details with a simple model based on an analysis of the momenta and kinetic energies of the colliding objects, called *Newton's law of restitution for instantaneous collisions with no friction*. It makes the following simplifying assumptions about the collision:

- The collision force acts over an infinitesimally short period of time, turning it into what we call an idealized *impulse*. This causes the velocities of the bodies to change *instantaneously* as a result of the collision.
- There is no friction at the point of contact between the objects' surfaces. This is another way of saying that the impulse acting to separate the bodies during the collision is normal to both surfaces—there is no tangential component to the collision impulse. (This is just an idealization of course; we'll get to friction in Section 13.4.7.5.)
- The nature of the complex submolecular interactions between the bodies during the collision can be approximated by a single quantity known as the *coefficient of restitution*, customarily denoted by the symbol ϵ . This coefficient describes how much energy is lost during the collision. When $\epsilon = 1$, the collision is perfectly elastic, and no energy is lost. (Picture two billiard balls colliding in mid-air.) When $\epsilon = 0$, the collision is perfectly *inelastic*, also known as perfectly *plastic* and the kinetic energy of both bodies is lost. The bodies will stick together after the collision, continuing to move in the direction that their mutual center of mass had been moving before the collision. (Picture pieces of putty being slammed together.)

All collision analysis is based around the idea that linear momentum is conserved. So for two bodies 1 and 2, we can write

$$\mathbf{p}_1 + \mathbf{p}_2 = \mathbf{p}'_1 + \mathbf{p}'_2, \quad \text{or} \\ m_1 \mathbf{v}_1 + m_2 \mathbf{v}_2 = m'_1 \mathbf{v}'_1 + m'_2 \mathbf{v}'_2$$

where the primed symbols represent the momenta and velocities after the collision. The kinetic energy of the system is conserved as well, but we must account for the energy lost due to heat and sound by introducing an additional energy loss term T_{lost} :

$$\frac{1}{2} m_1 v_1^2 + \frac{1}{2} m_2 v_2^2 = \frac{1}{2} m'_1 v'^2_1 + \frac{1}{2} m'_2 v'^2_2 + T_{\text{lost}}.$$

If the collision is perfectly *elastic*, the energy loss T_{lost} is zero. If it is perfectly *plastic*, the energy loss is equal to the original kinetic energy of the system, the primed kinetic energy sum becomes zero and the bodies stick together after the collision.

To resolve a collision using Newton's law of restitution, we apply an idealized *impulse* to the two bodies. An impulse is like a force that acts over an infinitesimally short period of time and thereby causes an instantaneous change in the velocity of the body to which it is applied. We could denote an impulse with the symbol $\Delta \mathbf{p}$, since it is a change in momentum ($\Delta \mathbf{p} = m \Delta \mathbf{v}$). However, most physics texts use the symbol $\hat{\mathbf{p}}$ (pronounced "p-hat") instead, so we'll do the same.

Because we assume that there is no friction involved in the collision, the impulse vector must be normal to both surfaces at the point of contact. In other words, $\hat{\mathbf{p}} = \hat{p} \mathbf{n}$, where $\mathbf{n}$ is the unit vector normal to both surfaces. This is illustrated in Figure 13.27. If we assume that the surface normal points toward body 1, then body 1 experiences an impulse of $\hat{\mathbf{p}}$, and body 2 experiences an equal but opposite impulse $-\hat{\mathbf{p}}$. Hence, the momenta of the two bodies after the collision can be written in terms of their momenta prior to the collision and the impulse $\hat{\mathbf{p}}$ as follows:

$$\begin{aligned} \mathbf{p}'_1 &= \mathbf{p}_1 + \hat{\mathbf{p}} & \mathbf{p}'_2 &= \mathbf{p}_2 - \hat{\mathbf{p}} \\ m_1 \mathbf{v}'_1 &= m_1 \mathbf{v}_1 + \hat{\mathbf{p}} & m_2 \mathbf{v}'_2 &= m_2 \mathbf{v}_2 - \hat{\mathbf{p}} \\ \mathbf{v}'_1 &= \mathbf{v}_1 + \frac{\hat{\mathbf{p}}}{m_1} \mathbf{n} & \mathbf{v}'_2 &= \mathbf{v}_2 + \frac{\hat{\mathbf{p}}}{m_2} \mathbf{n}. \end{aligned} \quad (13.9)$$

The *coefficient of restitution* provides the key relationship between the relative velocities of the bodies before and after the collision. Given that the centers of mass of the bodies have velocities before the collision and afterward, the coefficient of restitution ε is defined as follows:

$$(\mathbf{v}'_2 - \mathbf{v}'_1) = \varepsilon(\mathbf{v}_2 - \mathbf{v}_1). \quad (13.10)$$

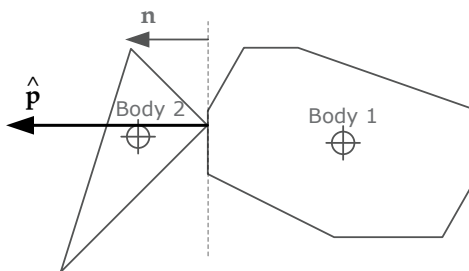


Figure 13.27. In a frictionless collision, the impulse acts along a line normal to both surfaces at the point of contact. This line is defined by the unit normal vector $\mathbf{n}$.

Solving Equations (13.9) and (13.10) under the temporary assumption that the bodies cannot rotate yields

$$\hat{\mathbf{p}} = \hat{p}\mathbf{n} = \frac{(\varepsilon + 1)(\mathbf{v}_2 \cdot \mathbf{n} - \mathbf{v}_1 \cdot \mathbf{n})}{\frac{1}{m_1} + \frac{1}{m_2}}\mathbf{n}.$$

Notice that if the coefficient of restitution is one (perfectly elastic collision) and if the mass of body 2 is effectively infinite (as it would be for, say, a concrete driveway), then $(1/m_2) = 0$, $\mathbf{v}_2 = 0$, and this expression reduces to a reflection of the other body's velocity vector about the contact normal, as we'd expect:

$$\begin{aligned}\hat{\mathbf{p}} &= -2m_1(\mathbf{v}_1 \cdot \mathbf{n})\mathbf{n}; \\ \mathbf{v}'_1 &= \frac{\mathbf{p}_1 + \mathbf{p}_2}{m_1} \\ &= \frac{m_1\mathbf{v}_1 - 2m_1(\mathbf{v}_1 \cdot \mathbf{n})\mathbf{n}}{m_1} \\ &= \mathbf{v}_1 - 2m_1(\mathbf{v}_1 \cdot \mathbf{n})\mathbf{n}.\end{aligned}$$

The solution gets a bit hairier when we take the rotations of the bodies into account. In this case, we need to look at the velocities of the points of contact on the two bodies rather than the velocities of their centers of mass, and we need to calculate the impulse in such a way as to impart a realistic rotational effect as a result of the collision. We won't get into the details here, but Chris Hecker's article, available at <http://chrishecker.com/images/e/e7/Gdmphys3.pdf>, does an excellent job of describing both the linear and the rotational aspects of collision response. The theory behind collision response is explained more fully in [17].

13.4.7.3 Penalty Forces

Another approach to collision response is to introduce imaginary forces called *penalty forces* into the simulation. A penalty force acts like a stiff damped spring attached to the contact points between two bodies that have just interpenetrated. Such a force induces the desired collision response over a short but finite period of time. Using this approach, the spring constant k effectively controls the duration of the interpenetration, and the damping coefficient b acts a bit like the restitution coefficient. When $b = 0$, there is no damping—no energy is lost, and the collision is perfectly elastic. As b increases, the collision becomes more plastic.

Let's take a brief look at some of the pros and cons of the penalty force approach to resolving collisions. On the positive side, penalty forces are easy to implement and understand. They also work well when three or more bodies are interpenetrating each other. This problem is very difficult to solve when resolving collisions one pair at a time. A good example is the Sony PS3 demo in which a huge number of rubber duckies are poured into a bathtub—the simulation was nice and stable despite the very large number of collisions. The penalty force method is a great way to achieve this.

Unfortunately, because penalty forces respond to penetration (i.e., relative position) rather than to relative velocity, the forces may not align with the direction we would intuitively expect, especially during a high-speed collision. A classic example is a car driving head-on into a truck. The car is low while the truck is tall. Using only the penalty force method, it is easy to arrive at a situation in which the penalty force is vertical, rather than horizontal as we would expect given the velocities of the two vehicles. This can cause the truck to pop its nose up into the air while the car drives under it.

In general, the penalty force technique works well for low-speed impacts, but it does not work well at all when objects are moving quickly. It is possible to combine the penalty force method with other collision resolution approaches in order to strike a balance between stability in the presence of large numbers of interpenetrations and responsiveness and more-intuitive behavior at high velocities.

13.4.7.4 Using Constraints to Resolve Collisions

As we'll investigate in Section 13.4.8, most physics systems permit various kinds of constraints to be imposed on the motion of the bodies in the simulation. If collisions are treated as constraints that disallow object interpenetration, then they can be resolved by simply running the simulation's general-purpose constraint solver. If the constraint solver is fast and produces high-

quality visual results, this can be an effective way to resolve collisions.

13.4.7.5 Friction

Friction is a force that arises between two bodies that are in continuous contact, resisting their movement relative to one another. There are a number of types of friction. *Static friction* is the resistance one feels when trying to start a stationary object sliding along a surface. *Dynamic friction* is a resisting force that arises when objects are actually moving relative to one another. *Sliding friction* is a type of dynamic friction that resists movement when an object slides along a surface. *Rolling friction* is a type of static or dynamic friction that acts at the point of contact between a wheel or other round object and the surface it is rolling on. When the surface is very rough, the rolling friction is exactly strong enough to cause the wheel to roll without sliding, and it acts as a form of static friction. If the surface is somewhat smooth, the wheel may slip, and a dynamic form of rolling friction comes into play. *Collision friction* is the friction that acts instantaneously at the point of contact when two bodies collide while moving. (This is the friction force that we ignored when discussing Newton's law of restitution in Section 13.4.7.1.) Various kinds of *constraints* can have friction as well. For example, a rusted hinge or axle might resist being turned by introducing a friction torque.

Let's look at an example to understand the essence of how friction works. Linear sliding friction is proportional to the component of an object's weight that is acting normal to the surface on which it is sliding. The weight of an object is just the force due to gravity, $\mathbf{G} = m\mathbf{g}$, which is always directed downward. The component of this force normal to an inclined surface that makes an angle θ with the horizontal is just $G_N = mg \cos \theta$. The friction force f is then

$$f = \mu mg \cos \theta,$$

where the constant of proportionality μ is called the *coefficient of friction*. This force acts tangentially to the surface, in a direction opposite to the attempted or actual motion of the object. This is illustrated in Figure 13.28.

Figure 13.28 also shows the component of the gravitational force acting tangent to the surface, $G_T = mg \sin \theta$. This force tends to make the object accelerate down the plane, but in the presence of sliding friction, it is counteracted by f . Hence, the net force tangent to the surface is

$$F_{\text{net}} = G_T - f = mg(\sin \theta - \mu \cos \theta).$$

If the angle of inclination is such that the expression in parentheses is zero, the object will slide at a constant speed (if already moving) or be at rest. If the

expression is greater than zero, the object will accelerate down the surface. If it is less than zero, the object will decelerate and eventually come to rest.

13.4.7.6 Welding

An additional problem arises when an object is sliding across a polygon soup. Recall that a polygon soup is just what its name implies—a soup of essentially unrelated polygons (usually triangles). As an object slides from one triangle to the next, the collision detection system will generate additional spurious contacts because it will think that the object is about to hit the edge of the next triangle. This is illustrated in Figure 13.29.

There are a number of solutions to this problem. One is to analyze the set of contacts and discard ones that appear to be spurious, based on various heuristics and possibly some knowledge of the object's contacts on a previous frame (e.g., if we know the object was sliding along a surface and a contact normal arises that is due to the object being near the edge of its current triangle, then discard that contact normal). Versions of Havok prior to 4.5 employed this approach.

Starting with Havok 4.5, a new technique was implemented that essentially annotates the mesh with triangle adjacency information. The collision detection system therefore “knows” which edges are interior edges and can discard spurious collisions reliably and quickly. Havok describes this solution as *welding*, because in effect the edges of the triangles in the poly soup are welded to one another.

13.4.7.7 Coming to Rest, Islands and Sleeping

When energy is removed from a simulated system via friction, damping or other means, moving objects will eventually come to rest. This seems like a natural consequence of the simulation—something that would just “fall out” of the differential equations of motion. Unfortunately, in a real computerized

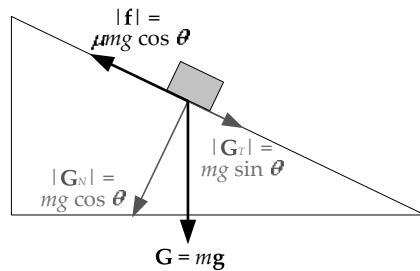


Figure 13.28. The force of friction f is proportional to the normal component of the object's weight. The proportionality constant μ is called the coefficient of friction.

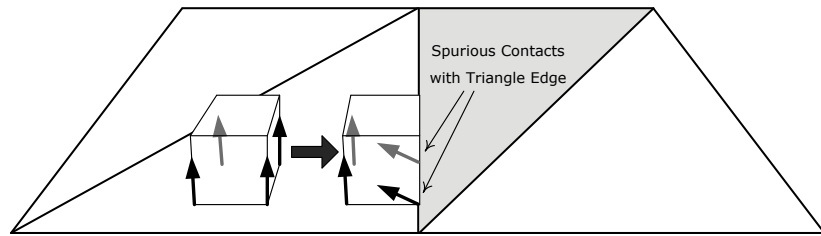


Figure 13.29. When an object slides between two adjacent triangles, spurious contacts with the new triangle's edge can be generated.

simulation, coming to rest is never quite that simple. Various factors such as floating-point error, inaccuracies in the calculation of restitution forces and numerical instability can cause objects to jitter forever rather than coming to rest as they should. For this reason, most physics engines use various heuristic methods to detect when objects are oscillating instead of coming to rest as they should. Additional energy can be removed from the system to ensure that such objects eventually settle down, or they can simply be stopped abruptly once their average velocity drops below a threshold.

When an object really does stop moving (finds itself in a state of equilibrium), there is no reason to continue integrating its equations of motion every frame. To optimize performance, most physics engines allow dynamic objects in the simulation to be *put to sleep*. This excludes them from the simulation temporarily, although sleeping objects are still active from a collision standpoint. If any force or impulse begins acting on a sleeping object, or if the object loses one of the contacts that was holding it in equilibrium, it will be awoken so that its dynamic simulation can be resumed.

Sleep Criteria

Various criteria can be used to determine whether or not a body qualifies for sleep. It's not always easy to make this determination in a robust manner for all situations. For example, a long pendulum might have very low angular momentum and yet still be moving visibly on-screen.

The most commonly used criteria for equilibrium detection include:

- The body is *supported*. This means it has three or more contact points (or one or more *planar* contacts) that allow it to attain equilibrium with gravity and any other forces that might be affecting it.
- The body's *linear and angular momentum* are below a predefined threshold.

- A *running average* of the linear and angular momentum are below a predefined threshold.
- The total *kinetic energy* of the body ($T = \frac{1}{2}\mathbf{p} \cdot \mathbf{v} + \frac{1}{2}\mathbf{L} \cdot \boldsymbol{\omega}$) is below a predefined threshold. The kinetic energy is usually mass-normalized so that a single threshold can be used for all bodies regardless of their masses.

The motion of a body that is about to go to sleep might be *progressively damped* so that it comes to a smooth stop rather than stopping abruptly.

Simulation Islands

Both Havok and PhysX further optimize their performance by automatically grouping objects that either are interacting or have the potential to interact in the near future into sets called *simulation islands*. Each simulation island can be simulated independently of all the other islands—an approach that is highly conducive to cache coherency optimizations and parallel processing.

Havok and PhysX both put entire islands to sleep rather than individual rigid bodies. This approach has its pros and cons. The performance boost is obviously larger when a whole group of interacting objects can be put to sleep. On the other hand, if even one object in an island is awake, the entire island is awake. Overall, it seems that the pros tend to outweigh the cons, so the simulation island design is one we're likely to continue to see in future versions of these SDKs.

13.4.8 Constraints

An unconstrained rigid body has six degrees of freedom (DOF): It can translate in three dimensions, and it can rotate about the three Cartesian axes. *Constraints* restrict an object's motion, reducing its degrees of freedom either partially or completely. Constraints can be used to model all sorts of interesting behaviors in a game. Here are a few examples:

- a swinging chandelier (point-to-point constraint);
- a door that can be kicked, slammed, blown off its hinges (hinge constraint);
- a vehicle's wheel assembly (axle constraint with damped springs for suspension);
- a train or a car pulling a trailer (stiff spring/rod constraint);
- a rope or chain (chain of stiff springs or rods); and

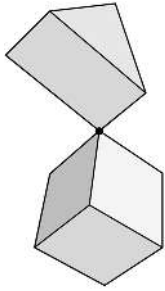


Figure 13.30. A point-to-point constraint requires that a point on body A aligns with a point on body B.

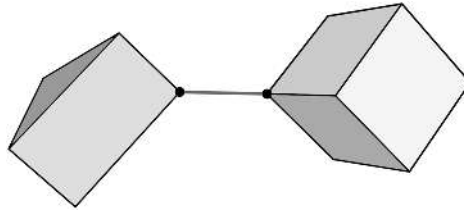


Figure 13.31. A stiff spring constraint requires that a point on body A be separated from a point on body B by a user-specified distance.

- a rag doll (specialized constraints that mimic the behavior of various joints in the human skeleton).

In the sections that follow, we'll briefly investigate these and some of the other most common kinds of constraints typically provided by a physics SDK.

13.4.8.1 Point-to-Point Constraints

A point-to-point constraint is the simplest type of constraint. It acts like a ball-and-socket joint—bodies can move in any way they like, as long as a specified point on one body lines up with a specified point on the other body. This is illustrated in Figure 13.30.

13.4.8.2 Stiff Springs

A stiff spring constraint is a lot like a point-to-point constraint except that it keeps the two points separated by a specified distance. This kind of constraint acts like an invisible rod between the two constrained points. Figure 13.31 illustrates this constraint.

13.4.8.3 Hinge Constraints

A hinge constraint limits rotational motion to only a single degree of freedom, about the hinge's axis. An *unlimited hinge* acts like an axle, allowing the constrained object to complete an unlimited number of full rotations. It's common to define *limited hinges* that can only move through a predefined range of angles about the one allowed axis. For example, a one-way door can only move through a 180 degree arc, because otherwise it would pass through the adjacent wall. Likewise, a two-way door is constrained to move through a ± 180 degree arc. Hinge constraints may also be given a degree of friction in the form of a torque that resists rotation about the hinge's axis. A limited hinge constraint is shown in Figure 13.32.

13.4.8.4 Prismatic Constraints

Prismatic constraints act like a piston: A constrained body's motion is restricted to a single translational degree of freedom. A prismatic constraint may or may not permit rotation about the translation axis of the piston. Prismatic constraints can of course be limited or unlimited and may or may not include friction. A prismatic constraint is illustrated in Figure 13.33.

13.4.8.5 Other Common Constraint Types

Many other types of constraints are possible, of course. Here are just a few examples:

- *Planar*. Objects are constrained to move in a two-dimensional plane.
- *Wheel*. This is typically a hinge constraint with unlimited rotation, coupled with some form of vertical suspension simulated via a spring-damper assembly.
- *Pulley*. In this specialized constraint, an imaginary rope passes through a pulley and is attached to two bodies. The bodies move along the line of the rope via a leverage ratio.

Constraints may be breakable, meaning that after enough force is applied, they automatically come apart. Alternatively, the game can turn the constraint on and off at will, using its own criteria for when the constraint should break.

13.4.8.6 Constraint Chains

Long chains of linked bodies are sometimes difficult to simulate in a stable manner because of the iterative nature of the constraint solver. A *constraint chain* is a specialized group of constraints with information that tells the constraint solver how the objects are connected. This allows the solver to deal with the chain in a more stable manner than would otherwise be possible.

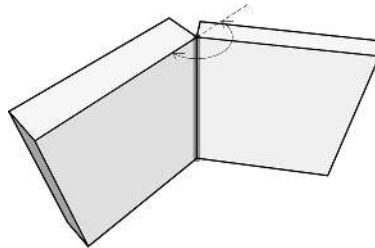


Figure 13.32. A limited hinge constraint mimics the behavior of a door.

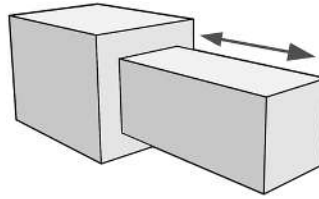


Figure 13.33. A prismatic constraint acts like a piston.

13.4.8.7 Rag Dolls

A rag doll is a physical simulation of the way a human body might move when it is dead or unconscious and hence entirely limp. Rag dolls are created by linking together a collection of rigid bodies, one for each semi-rigid part of the body. For example, we might have capsules for the feet, calves, thighs, hands, upper and lower arms and head and possibly a few for the torso to simulate the flexibility of the spine.

The rigid bodies in a rag doll are connected to one another via constraints. Rag doll constraints are specialized to mimic the kinds of motions the joints in a real human body can perform. We usually make use of constraint chains to improve the stability of the simulation.

A rag doll simulation is always tightly integrated with the animation system. As the rag doll moves in the physics world, we extract the positions and rotations of the rigid bodies, and we use this information to drive the positions and orientations of certain joints in the animated skeleton. So in effect, a rag doll is really just a form of *procedural animation* that happens to be driven by the physics system. (See Chapter 12 for more details on skeletal animation.)

Of course, implementing a rag doll is not quite as simple as I've made it sound here. For one thing, there's usually not a one-to-one mapping between the rigid bodies in the rag doll and the joints in the animated skeleton—the skeleton usually has more joints than the rag doll has bodies. Therefore, we need a system that can map rigid bodies to joints (i.e., one that “knows” to which joint each rigid body in the rag doll corresponds). There may be additional joints between those that are being driven by the rag doll bodies, so the mapping system must also be capable of determining the correct pose transforms for these intervening joints. This is not an exact science. We must apply artistic judgment and/or some knowledge of human biomechanics in order to achieve a natural-looking rag doll.

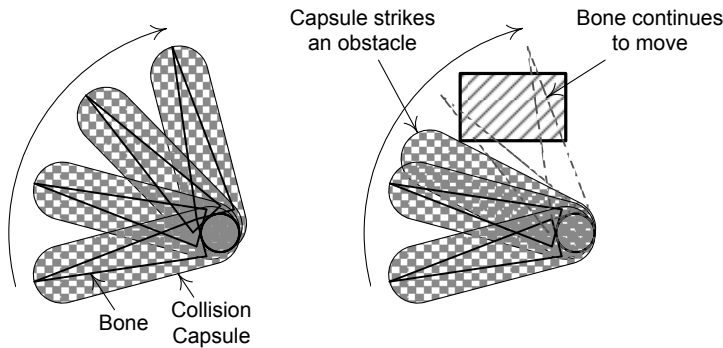


Figure 13.34. With a powered rag doll constraint, and in the absence of any additional forces or torques, a rigid body representing the lower arm can be made to exactly track the movements of an animated elbow joint (left). If an obstacle blocks the motion of the body, it will diverge from that of the animated elbow joint in a realistic way (right).

13.4.8.8 Powered Constraints

Constraints can also be “powered,” meaning that an external engine system such as the animation system can indirectly control the translations and orientations of the rigid bodies in the rag doll.

Let’s take an elbow joint as an example. An elbow acts pretty much like a limited hinge, with a little less than 180 degrees of free rotation. (Actually, an elbow can also rotate axially, but we’ll ignore that for the purposes of this discussion.) To power this constraint, we model the elbow as a *rotational spring*. Such a spring exerts a torque proportional to the spring’s angle of deflection away from some predefined rest angle, $N = -k(\theta - \theta_{\text{rest}})$. Now imagine changing the rest angle externally, say by ensuring that it always matches the angle of the elbow joint in an animated skeleton. As the rest angle changes, the spring will find itself out of equilibrium, and it will exert a torque that tends to rotate the elbow back into alignment with θ_{rest} . In the absence of any other forces or torques, the rigid bodies will exactly track the motion of the elbow joint in the animated skeleton. But if other forces are introduced (for example, the lower arm comes in contact with an immovable object), then these forces will play into the overall motion of the elbow joint, allowing it to diverge from the animated motion in a somewhat realistic manner. As illustrated in Figure 13.34, this provides the illusion of a human who is trying her best to move in a certain way (i.e., the “ideal” motion provided by the animation) but who is sometimes unable to do so due to the limitations of the physical world (e.g., her arm gets caught on something as she tries to swing it forward).

13.4.9 Controlling the Motions of Rigid Bodies

Most game designs call for a degree of control over the way rigid bodies move over and above the way they would move naturally under the influence of gravity and in response to collisions with other objects in the scene. For example:

- An air vent applies an upward force to any object that enters its shaft of influence.
- A car is coupled to a trailer and exerts a pulling force on it as it moves.
- A tractor beam exerts a force on an unwitting spacecraft.
- An anti-gravity device causes objects to hover.
- The flow of a river creates a force field that causes objects floating in the river to move downstream.

And the list goes on. Most physics engines typically provide their users with a number of ways to exert control over the bodies in the simulation. We'll outline the most common of these mechanisms in the following sections.

13.4.9.1 Gravity

Gravity is ubiquitous in most games that take place on the surface of the Earth or some other planet (or on a spacecraft with simulated gravity). Gravity is technically not a force but rather a (roughly) constant acceleration, so it affects all bodies equally regardless of their mass. Because of its ubiquitous and special nature, the magnitude and direction of the gravitational acceleration is specified via a global setting in most SDKs. (If you're writing a space game, you can always set gravity to zero to eliminate it from the simulation.)

13.4.9.2 Applying Forces

Any number of forces can be applied to the bodies in a game physics simulation. A force always acts over a finite time interval. (If it acted instantaneously, it would be called an *impulse*—more on that in Section 13.4.9.4 below.) The forces in a game are often dynamic in nature—they often change their directions and/or their magnitudes every frame. So the force-application function in most physics SDKs is designed to be called once per frame for the duration of the force's influence. The signature of such a function usually looks something like this: `applyForce(const Vector& forceInNewtons)`, where the duration of the force is assumed to be Δt .

13.4.9.3 Applying Torques

When a force is applied such that its line of action passes through the center of mass of a body, no torque is generated, and only the body's linear acceleration is affected. If it is applied off-center, it will induce *both* a linear and a rotational acceleration. A *pure torque* can be applied to a body as well by applying two equal and opposite forces to points equidistant from the center of mass. The linear motions induced by such a pair of forces will cancel each other out (since for the purposes of linear dynamics, the forces both act at the center of mass). This leaves only their rotational effects. A pair of torque-inducing forces like this is known as a *couple* ([http://en.wikipedia.org/wiki/Couple_\(mechanics\)](http://en.wikipedia.org/wiki/Couple_(mechanics))). A special function such as `applyTorque(const Vector& torque)` may be provided for this purpose. However, if your physics SDK provides no `applyTorque()` function, you can always write one and have it generate a suitable couple instead.

13.4.9.4 Applying Impulses

As we saw in Section 13.4.7.2, an *impulse* is an instantaneous change in velocity (or actually, a change in momentum). Technically speaking, an impulse is a force that acts for an infinitesimal amount of time. However, the shortest possible duration of force application in a time-stepped dynamics simulation is Δt , which is not short enough to simulate an impulse adequately. As such, most physics SDKs provide a function with a signature such as `applyImpulse(const Vector& impulse)` for the purposes of applying impulses to bodies. Of course, impulses come in two flavors—linear and angular—and a good SDK should provide functions for applying both types.

13.4.10 The Collision/Physics Step

Now that we've covered the theory and some of the technical details behind implementing a collision and physics system, let's take a brief look at how these systems actually perform their updates every frame.

Every collision/physics engine performs the following basic tasks during its update step. Different physics SDKs may perform these phases in different orders. That said, the technique I've seen used most often goes something like this:

1. The forces and torques acting on the bodies in the physics world are integrated forward by Δt in order to determine their tentative positions and orientations next frame.

2. The collision detection library is called to determine if any new contacts have been generated between any of the objects as a result of their tentative movement. (The bodies normally keep track of their contacts in order to take advantage of temporal coherency. Hence, at each step of the simulation, the collision engine need only determine whether any previous contacts have been lost and whether any new contacts have been added.)
3. Collisions are resolved, often by applying impulses or penalty forces or as part of the constraint-solving step below. Depending on the SDK, this phase may or may not include continuous collision detection (CCD, otherwise known as time of impact detection or TOI).
4. Constraints are satisfied by the constraint solver.

At the conclusion of step 4, some of the bodies may have moved away from their tentative positions as determined in step 1. This movement may cause additional interpenetrations between objects or cause other previously satisfied constraints to be broken. Therefore, steps 1 through 4 (or sometimes only 2 through 4, depending on how collisions and constraints are resolved) are repeated until either (a) all collisions have been successfully resolved and all constraints are satisfied, or (b) a predefined maximum number of iterations has been exceeded. In the latter case, the solver effectively “gives up,” with the hope that things will resolve themselves naturally during subsequent frames of the simulation. This helps to avoid performance spikes by amortizing the cost of collision and constraint resolution over multiple frames. However, it can lead to incorrect-looking behavior if the errors are too large or if the time step is too long or is inconsistent. Penalty forces can be blended into the simulation in order to gradually resolve these problems over time.

13.4.10.1 The Constraint Solver

A constraint solver is essentially an iterative algorithm that attempts to satisfy a large number of constraints simultaneously by *minimizing the error* between the actual positions and rotations of the bodies in the physics world and their ideal positions and rotations as defined by the constraints. As such, constraint solvers are essentially iterative error-minimization algorithms.

Let’s take a look first at how a constraint solver works in the trivial case of a single pair of bodies connected by a single hinge constraint. During each step of the physics simulation, the numerical integrator will find new tentative transforms for the two bodies. The constraint solver then evaluates their

relative positions and calculates the error between the positions and orientations of their shared axis of rotation. If any error is detected, the solver moves the bodies in such a way as to minimize or eliminate it. Since there are no other bodies in the system, the second iteration of the step should discover no new contacts, and the constraint solver will find that the one hinge constraint is now satisfied. Hence the loop can exit without further iterations.

When more than one constraint must be satisfied simultaneously, more iterations may be required. During each iteration, the numerical integrator will sometimes tend to move the bodies out of alignment with their constraints, while the constraint solver tends to put them back into alignment. With luck, and a carefully designed approach to minimizing error in the constraint solver, this feedback loop should eventually settle into a valid solution. However, the solution may not always be exact. This is why, in games with physics engines, you sometimes witness seemingly impossible behaviors, like chains that stretch (opening up little gaps between the links), objects that interpenetrate briefly or hinges that momentarily move beyond their allowable ranges. The goal of the constraint solver is to minimize error—it's not always possible to eliminate it completely.

13.4.10.2 Variations between Engines

The description given above is of course an over-simplification of what really goes on in a physics/collision engine every frame. The way in which the various phases of computation are performed, and their order relative to one another, may vary from physics SDK to physics SDK. For example, some kinds of constraints are modeled as forces and torques that are taken care of by the numerical integration step rather than being resolved by the constraint solver. Collision may be run before the integration step rather than after. Collisions may be resolved in any number of different ways. Our goal here is merely to give you a taste of how these systems work. For a detailed understanding of how any one SDK operates, you'll want to read its documentation and probably also inspect its source code (presuming the relevant bits are available for you to read). The curious and industrious reader can get a good start by downloading and experimenting with Open Dynamics Engine (ODE) and/or PhysX, as these two SDKs are available for free. You can also learn a great deal from ODE's wiki, which is available at http://opende.sourceforge.net/wiki/index.php/Main_Page.